CPA Uncertainty Code: Issues I Found

Review of `Test_MLE_compute_CPA`, `MLE_compute_cpa_2D_PV` and `MLE_compute_cpa_2D_PVA` | October 27, 2026

The derivation and test code were developed with help from Claude (an AI assistant); I ran the MATLAB tests myself.

What I was checking

The program finds the Closest Point of Approach (CPA) between a sensor and a contact and reports how uncertain the CPA time, bearing and range are. Those uncertainties need *partial derivatives* (how much each result moves when an input moves). The existing code computes some of them numerically. I derived them algebraically and compared the two.

Bottom line

The CPA time, bearing and range agree exactly between the existing code and my algebra. The **range** uncertainty also agrees. The **bearing** and **time** uncertainties do not, and the evidence points to the existing code, not the algebra. I checked the algebra against a very fine numerical test and it matched.

	Existing code	My algebra	Difference
Time uncertainty (s)	97.45	100.41	existing 3% low
Bearing uncertainty (rad)	0.197	0.125	existing 58% high
Range uncertainty (m)	993.86	993.86	none

Issues I found in the code

1. Bearing uncertainty ignores that the CPA time moves. **Likely** (CA function), **Confirmed** (CV function)

When the inputs change, the CPA time shifts, and bearing changes quickly in that direction. The CV function's bearing formula treats the CPA time as fixed. If the accelerating function does the same, the bearing uncertainty comes out too large: 0.197 rad (11.3°) instead of 0.125 rad (7.1°). My evidence is that the fixed-time version of my algebra reproduces the existing number to six digits (0.196654). *Fix:* include the CPA-time term.

2. Time uncertainty is probably computed with too large a step. **Likely**

A numerical partial nudges an input and watches the result. The acceleration uncertainty is only 0.001, and a nudge of about 0.001 reproduces the existing time value (97.449) almost exactly. That is as large as the uncertainty itself, so the estimate is biased and unstable: in my Python tests a step of 0.003 gave 580 to 1080 s instead of 100 s. *Fix:* use the algebraic formula, which has no step size.

3. Different methods are mixed in one function. **Likely**

Range matches my algebra exactly, while time and bearing do not. That suggests range is algebraic, time is numerical, and bearing leaves out a term. *Fix:* use one consistent method for all three.

4. Silent caps on uncertainties (CV function only). **Confirmed**

Time uncertainty is capped at 3600 s, bearing at $\pi/2$, range at 10000 m, with no message. The

accelerating function has no caps, so the two cases behave differently. *Fix:* one policy, with a flag when a cap is applied.

5. A CPA more than 2 hours away returns empty outputs. **Confirmed**
The CV function returns `[]` for everything. Callers that do arithmetic on the result (for example `CPA_time_CV/60`) will print blanks or fail. *Fix:* return `NaN`, or have the caller test `isempty`.

6. Results depend on hand-edited settings. **Confirmed**
`Phat` for the accelerating case is chosen by commenting lines in and out. With the wrong line active the answers change silently (a velocity std of 1.0 instead of 0.1 gives a time uncertainty of 162 s instead of 100 s). *Fix:* set `Phat` by named test case and print it.

7. Minor housekeeping. **Confirmed**
MATLAB flags the `= []` initializations as unused, there is commented-out debug code, and the documentation calls the inputs 3-D while the outputs are "2D range." None of this changes results.

What is and is not proven

- **Confirmed**: the CPA time, bearing and range agree; my algebraic partials agree with a fine numerical check; the CV bearing formula holds the CPA time fixed.
- **Likely**: issues 1–3 for the accelerating function. They fit the numbers closely, but I have not yet read `MLE_compute_cpa_Phi.m`, the file that computes those partials.
- All results are for one scenario. I have not tried near-collision, past-CPA or large-acceleration cases.

Next steps

1. Read `MLE_compute_cpa_Phi.m` to confirm issues 1–3.
2. Swap in the algebraic partials (`MLE_compute_cpa_Phi_analytic.m`) and run `Test_analytic_partials`.
3. Agree on a bearing convention (I recommend the full derivative) and a policy for caps and out-of-range CPAs.
4. Rerun on more scenarios before relying on the results.

Appendix: the test code

Where each test was run.

Test	Where	Result
A. Analytic function vs existing code	MATLAB (I ran it)	Range matches; time and bearing differ
B. Finite difference of my function	MATLAB (I ran it)	100.415 and 0.124675: algebra is self-consistent
C. CPA time, bearing, range comparison	MATLAB (I ran it)	All three match
D. Fixed-time bearing check	Python only	0.196654, matches existing code
E. Step-size scan of time uncertainty	Python only	Forward step 1e-3 gives 97.448
F. `Test_analytic_partials.m`	Not run yet	Comparison script for the new files

A. Analytic function (`MLE_compute_cpa_2D_PVA_analytic.m`), called from the test script as `[t2,b2,r2,st2,sb2,s = MLE_compute_cpa_2D_PVA_analytic(Rpos,Rvel,Racc,Phat);`

```
function [t,brg,rng,sig_t,sig_b,sig_r,J] = MLE_compute_cpa_2D_PVA_analytic(Rpos,Rvel,Racc,Phat)
p = Rpos(1:2); v = Rvel(1:2); a = Racc(1:2); % 2D only (z = 0 in the tests)
p=p(:); v=v(:); a=a(:);
c = [0.5*(a'*a), 1.5*(v'*a), v'*v + p'*a, p'*v]; % cubic: R.V = 0
rts = roots(c);
rts = real(rts(abs(imag(rts)) < 1e-9));
best = inf; t = NaN;
for k = 1:numel(rts)
tk = rts(k);
Rk = p + v*tk + 0.5*a*tk^2; Vk = v + a*tk;
if Vk'*Vk + Rk'*a > 0 && norm(Rk) < best % range minimum
best = norm(Rk); t = tk;
end
end
R = p + v*t + 0.5*a*t^2; V = v + a*t; r = norm(R);
D = V'*V + R'*a;
I2 = eye(2);
dRdth = [I2, t*I2, 0.5*t^2*I2]; % 2x6, t held fixed
dfdth = [V', (t*V+R)', (0.5*t^2*V + t*R)']; % 1x6
dtdth = -dfdth / D; % implicit differentiation
dRtot = dRdth + V*dtdth; % total derivative of R(t*)
J = [dtdth;
[R(2), -R(1)]/r^2 * dRtot; % bearing = atan2(Rx,Ry)
(R'/r) * dRtot]; % range
S = J*Phat*J';
sig_t = sqrt(S(1,1)); sig_b = sqrt(S(2,2)); sig_r = sqrt(S(3,3));
brg = atan2(R(1),R(2)); rng = r;
end
```

Comparison printed after the test script (test A):

```
fprintf('time␣std:␣%g␣vs␣%g\nbrg␣std:␣%g␣vs␣%g\nrng␣std:␣%g␣vs␣%g\n', ...
CPA_time_std_CA,st2, CPA_brg_std_CA,sb2, CPA_rng_std_CA,sr2);
```

B. Finite difference of my own function (checks that the algebra matches its own numerical derivative):

```
fn = @MLE_compute_cpa_2D_PVA_analytic;
th0 = [Rpos(1:2)'; Rvel(1:2)'; Racc(1:2)']; % [px py vx vy ax ay]'
Jn = zeros(2,6);
for k = 1:6
```

```
h = 1e-5*max(abs(th0(k)), 1e-3);
d = zeros(6,1); d(k) = h;
tp = th0 + d; tm = th0 - d;
[t_p,b_p] = fn(tp(1:2),tp(3:4),tp(5:6),Phat);
[t_m,b_m] = fn(tm(1:2),tm(3:4),tm(5:6),Phat);
Jn(:,k) = [t_p - t_m; b_p - b_m]/(2*h);
end
S = Jn*Phat*Jn';
fprintf('FD␣of␣my␣function:␣time␣std␣%g,␣brg␣std␣%g\n', sqrt(S(1,1)), sqrt(S(2,2)));
```

C. Same CPA solution check (uses the existing solver):

```
[T1,B1,R1] = MLE_compute_cpa_acc(Rpos,Rvel,Racc);
fprintf('yours:␣␣%g␣min,␣%g␣deg,␣%g␣km\n', T1/60, B1*180/pi, R1/1000);
fprintf('mine:␣␣␣%g␣min,␣%g␣deg,␣%g␣km\n', t2/60, b2*180/pi, r2/1000);
```

D. Fixed-time bearing check (MATLAB version, to be run; the Python version below gave 0.196654):

```
p=Rpos(1:2)'; v=Rvel(1:2)'; a=Racc(1:2)';
R = p + v*t2 + 0.5*a*t2^2; r = norm(R); I2 = eye(2);
Jb = [R(2), -R(1)]/r^2 * [I2, t2*I2, 0.5*t2^2*I2];
fprintf('brg␣std␣without␣t*␣term:␣%g␣(yours:␣%g)\n', sqrt(Jb*Phat*Jb'), CPA_brg_std_CA);
```

F. Newer comparison script. `Test_analytic_partials.m` (supplied separately, with `MLE_compute_cpa_Phi_anal` compares the existing partials, the algebraic partials with both bearing conventions, and a fine finite difference of the solution itself. I have not run it yet.